## 2.7. Seguridad

### 2.7.1. Autenticación y gestión de sesiones

El acceso al panel web se realiza mediante correo electrónico y contraseña a través de `/api/login`. El servidor consulta la cuenta y exige que se encuentre activa y no esté eliminada. La contraseña recibida se compara con el hash almacenado mediante bcrypt; las contraseñas no se almacenan en texto plano. Cuando la cuenta no cumple las condiciones o la contraseña no coincide, se devuelve una respuesta HTTP 401 con un mensaje genérico, sin indicar cuál de las credenciales es incorrecta. La ruta de inicio de sesión limita las solicitudes a diez intentos por minuto mediante el middleware de control de frecuencia; al exceder el límite responde con HTTP 429.

Después de una autenticación válida, el servidor emite un token JWT firmado con el algoritmo HS256 y una vigencia de ocho horas. El token identifica al usuario e incluye la versión de su sesión. En cada petición protegida, el middleware comprueba la firma y la expiración, consulta nuevamente la cuenta y compara la versión del token con `version_sesion`. También verifica que el usuario permanezca activo y pertenezca a la clínica solicitada.

La interfaz conserva el token en `sessionStorage`, bajo la clave `clinica-token`. El interceptor HTTP lo incorpora al encabezado `Authorization` utilizando el esquema Bearer. Este almacenamiento está asociado a la sesión de la pestaña y no se utiliza `localStorage` para conservar el token. Sin embargo, `sessionStorage` es accesible desde JavaScript: no equivale a una cookie `HttpOnly` ni proporciona protección por sí solo frente a la ejecución de scripts maliciosos.

La implementación no utiliza tokens de refresco ni renovación automática. Cuando el JWT vence, la API responde con HTTP 401; el interceptor elimina el token almacenado y dirige al usuario a la pantalla de acceso para que inicie sesión nuevamente. Esta conducta también se aplica cuando el servidor detecta una sesión invalidada.

El cierre de sesión llama a `/api/logout`. El servidor incrementa `version_sesion`, de modo que los tokens previamente emitidos para esa cuenta dejan de ser válidos en las siguientes solicitudes, incluso si todavía no han alcanzado su fecha de expiración. Después de recibir la confirmación, la interfaz limpia el almacenamiento de sesión y vuelve al inicio de sesión. Esta invalidación afecta a las sesiones anteriores de la cuenta, no solamente a la pestaña que solicita el cierre. El cambio y el restablecimiento de contraseña también incrementan la versión de sesión.

La clave de firma del JWT y las credenciales de base de datos y almacenamiento se obtienen desde variables de entorno del backend. El archivo local `backend/.env` está excluido del repositorio y estas credenciales no se incorporan al código del frontend.

### 2.7.2. Autorización por roles o perfiles

La autorización se aplica en dos niveles. En la interfaz se muestran las opciones y acciones disponibles según la sesión. En el servidor se comprueban los permisos de cada operación y el alcance de los registros solicitados. La comprobación del backend es necesaria aunque se invoque directamente una ruta HTTP sin utilizar el menú de la aplicación.

Las asociaciones de usuarios y roles se conservan en `roles_modelos`; los permisos de cada rol se almacenan en `permisos_roles`, y los permisos directos de usuarios en `permisos_modelos`. El servidor recupera esta información al validar las solicitudes. Los permisos distinguen operaciones como lectura, creación, actualización y eliminación. Los perfiles internos tienen las siguientes responsabilidades:

- **Administrador:** administra cuentas, roles y permisos y supervisa los registros del sistema dentro de las clínicas a las que pertenece. El rol interno `Super Admin` satisface las comprobaciones generales de permisos; las rutas de administración utilizan además la comprobación `requireAdmin`.
- **Odontólogo:** gestiona los pacientes que tiene asignados y sus solicitudes radiográficas, de acuerdo con los permisos otorgados. Las consultas de pacientes y radiografías comprueban la relación de asignación, y las consultas de citas y recetas incorporan también el identificador del profesional. No dispone de administración de cuentas y permisos por el hecho de tener el rol de odontólogo.

- **Recepcionista:** consulta, registra y actualiza datos administrativos de pacientes; consulta odontólogos y horarios; gestiona citas y asigna pacientes a profesionales. El servidor limita los campos visibles y modificables y no le concede acceso a radiografías, antecedentes clínicos, recetas o administración de cuentas. La revocación de una asignación está reservada al administrador.

CERPAX no es un perfil con inicio de sesión en el panel. Su acceso se limita al portal de entrega mediante un token temporal vinculado a una solicitud. La validación exige que el enlace esté vigente, no haya sido revocado ni utilizado y corresponda a una solicitud pendiente. El token se conserva como hash en la base de datos. Un enlace inválido o no disponible genera HTTP 410. El paciente se representa como registro clínico y no como actor de acceso al panel descrito en esta solución.

Para el aislamiento entre clínicas, el servidor verifica que el valor del encabezado `X-Company-ID` pertenezca a las clínicas autorizadas del usuario. Posteriormente incorpora ese alcance en las consultas. Los permisos insuficientes generan HTTP 403; algunos accesos a registros fuera del alcance autorizado responden con HTTP 404 para no revelar su existencia. El acceso local de desarrollo está condicionado a la configuración y a la dirección de conexión; se deshabilita cuando el entorno es producción.

### 2.7.3. Validación de entradas y protección

La interfaz utiliza controles de formulario para orientar la captura de datos, como campos obligatorios, tipos de entrada y opciones disponibles. El backend realiza su propia validación, ya que los datos pueden enviarse sin pasar por la interfaz. Las reglas comprueban campos permitidos, identificadores numéricos, formatos de correo, longitudes máximas, valores de listas, números finitos y estructuras JSON. Las validaciones clínicas comprueban fechas y horarios y rechazan intervalos inconsistentes y superposiciones de citas.

Las operaciones con PostgreSQL utilizan consultas parametrizadas: los valores recibidos se envían separados del texto SQL mediante marcadores como `$1` y `$2`. Los nombres de módulos y campos se resuelven desde el catálogo y los metadatos permitidos, y los identificadores SQL se validan antes de incorporarse a las consultas dinámicas. Estas medidas reducen el riesgo de inyección SQL sin atribuir a una consulta parametrizada la validación de nombres de tablas o columnas.

Para las radiografías se admiten PDF, PNG, JPEG y WebP con un tamaño máximo de 50 MiB. En la carga directa a Supabase Storage, la API valida el nombre, la extensión, el tipo MIME y el tamaño declarado antes de emitir la URL firmada. Al confirmar, comprueba la existencia del objeto, su tamaño y la correspondencia de su ruta con el acceso temporal. En las rutas de carga multipart que reciben el archivo en el servidor se utiliza Multer para limitar el tamaño y se identifica el formato a partir de la firma binaria. La validación de metadatos de la carga directa no se presenta como una inspección completa del contenido del archivo.

Los objetos radiográficos se mantienen en almacenamiento privado. Antes de abrir un archivo, la API vuelve a comprobar la autorización del usuario sobre la solicitud. Las operaciones que completan una entrega utilizan transacciones y restricciones de unicidad para conservar la coherencia entre la solicitud, el archivo y el acceso utilizado.

Express aplica los encabezados de seguridad de Helmet y una política de seguridad de contenido que restringe los orígenes de scripts, imágenes y conexiones. La API comprueba el encabezado de origen cuando está presente y rechaza los orígenes no permitidos con HTTP 403. Este control complementa la autenticación y no la sustituye. El procesamiento de cuerpos JSON está limitado a 2 MiB, de forma independiente del límite de archivos.

El manejador centralizado de errores devuelve respuestas JSON y evita exponer consultas internas, contraseñas o cadenas de conexión. Se utiliza HTTP 400 para errores de formato de la petición, HTTP 401 para autenticación, HTTP 403 para autorización, HTTP 409 para conflictos y HTTP 422 para validaciones de datos, según la ruta y la condición detectada. Los errores internos generan un mensaje genérico con HTTP 500. Las respuestas de la API incluyen `Cache-Control: no-store` para evitar su almacenamiento en cachés HTTP.

Estas medidas se respaldan con pruebas del backend que comprueban ausencia de sesión, permisos insuficientes, acceso entre clínicas, pacientes no asignados, enlaces vencidos o revocados y validación

de archivos y datos. Los resultados de cada escenario deben acompañarse de las evidencias correspondientes en la sección de pruebas.